

C15 — Prioritization and Tradeoffs

1. What interviewers test

"You have five features and capacity for two. Which do you build?" and "The deadline just moved up by a month. What do you cut?" are execution questions, and they are answered badly in a predictable way: the candidate names a framework (RICE, impact versus effort), assigns numbers with no source, and announces the top score as the answer. The interviewer is not testing whether you know acronyms. They are testing whether you can turn a goal into criteria, weigh options against those criteria with evidence, name what you give up, and defend a decision while staying open to changing it.

Where it shows up. **Inferred** from the round structures candidates commonly report: "which feature first," "what would you cut," stakeholder-conflict and resource-conflict prompts at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, often in execution rounds and often as a follow-up to a design or improvement answer (the interviewer asks, "you proposed three things; which first?") **[unverified: candidate-reported]**. On the job it is the weekly work of a PM: a backlog that is always larger than capacity, and every choice a statement about what the team values.

What is scored. Whether the decision follows from a stated goal **Inferred**. Interviewers listen for criteria derived from the goal, not borrowed from a template; for options sized with a reasoned estimate and an honest range; for tradeoffs named in terms of what is lost (not only "pros and cons"); for the riskiest assumption identified and tested early; for a decision, including what you will not do; and for a readiness to change the ranking when a better argument or new evidence arrives.

Six observable behaviors:

1. You state the goal and the constraints (capacity, deadline, dependencies) before ranking anything.
2. You derive the criteria from the goal and say how you weight them and why.
3. You size the options with a chain of reasoning, and give a range or a sensitivity, not a single score.
4. You name the tradeoffs and the opportunity cost of the choice, including the short-term against the long-term.
5. You identify the riskiest assumption and propose the cheapest test, or a minimum viable product, before committing.
6. You decide, say what you will not do, and say what would change your mind; and when you must refuse, you say no with a reason and an alternative.

The failure this prevents. Six patterns end prioritization answers: a framework used as a decision machine ([framework-theater], C03), criteria that do not come from the goal, scores with invented precision, a list of pros and cons with no decision, building the whole thing when a test would do, and a "no" with no reason or alternative.

2. The concept from first principles

C15.1 Prioritization as a reasoned decision

Prioritization is choosing what to do first under limited capacity, using stated criteria and acknowledged tradeoffs. Three things make it hard. There are always more ideas than capacity. The value of each idea is uncertain. And every choice has an **opportunity cost**: the value of the best alternative given up when a choice is made. Choosing feature A means not building B, and the true cost of A includes what B would have brought.

What is being decided. Four things, in order: the *goal* (what are we trying to achieve in this period?), the *criteria* (what makes an option good for that goal?), the *evidence* (what do we know about each option, and how sure are we?) and the *tradeoffs* (what do we give up?). A framework helps organize the last three and can never supply the first.

When not to use a full prioritization process. If there is one clear option, just do it. If the decision is cheap and reversible ([reversible-decision], C02), decide quickly and learn: a long analysis costs more than a wrong choice. If the real problem is not capacity but alignment (people disagree on the goal), no scoring will help; resolve the goal first (C32). If the options are not comparable until you learn more, run a test first (C15.6).

Counterexample. Asked which of five features to build, a candidate says: "I'd use RICE. Feature B scores 45,000, the highest, so build B." The interviewer asks what the team's goal is. The candidate has not said. If the goal is new-user activation, B (a feature mostly used by existing users) may be nearly irrelevant, and the score of 45,000 only shows that its reach is large. Worked example 1 shows the ranking changing when the same scoring is anchored to the goal. The framework produced a number; it did not make a decision.

Hidden risks. Two. *False precision*: a score like 45,000 looks exact and rests on guesses for reach, impact and confidence. *Gaming*: when scores decide, people inflate their favorite's confidence and shrink its effort; the process rewards persuasion and not truth. Both are why this chapter treats scoring as a way to organize thinking and test sensitivity.

C15.2 Frameworks: when each works and when it fails

Several frameworks are in common use. FW-C15-02 is a chooser; FW-C15-03 covers RICE in detail. None is better in general; each fits some situations.

Framework	What it does	Works when	Fails when
RICE (reach × impact × confidence ÷ effort)	Gives a comparable score per unit effort	Many similar options for one goal, with data on reach	Reach is not tied to the goal; impact is a guess; scores are gamed
ICE (impact, confidence, ease)	A quick, lightweight score	Fast triage of experiments or small ideas	Used for big, strategic bets; no reach
Value versus effort (2×2)	Shows quick wins and expensive bets	A workshop, to start a conversation	Values and efforts are unsized; everything lands in "quick win"
Weighted scoring	Scores options against weighted criteria	Criteria differ from simple impact/effort (e.g., strategy fit, risk)	Weights are arbitrary and tuned to a preferred result
MoSCoW (must, should, could, won't)	Sorts scope for a fixed deadline	Scoping a release; negotiating	Everything is a "must"
Kano model	Separates needs (must-haves), performance and delighters	Understanding what customers expect versus value	Used as a score; needs customer research
Cost of delay (and its weighted form, WSJF)	Ranks by value lost per unit of delay, per unit effort	Time-sensitive items with different urgency profiles	Delay costs cannot be estimated
Opportunity scoring	Importance versus satisfaction gaps from customers	Customer-research-rich contexts	No survey data

The rule. Use a framework to structure the discussion and expose assumptions, not to produce the answer. Whatever framework you use, three questions keep it honest: where did each number come from, what happens to the ranking if the least certain number changes (FW-C15-08), and does the top item serve the goal? A framework used without those questions is [[framework-theater]].

Why it works. A chooser prevents "RICE for everything." **When not to use any of them.** When the decision is obvious from the goal and a single constraint, say so in a sentence.

C15.3 Deriving criteria from goals

Criteria that come from a template ("impact, effort, confidence") are generic. Criteria derived from the goal are specific, and the specificity is what makes the decision defensible. FW-C15-04 is the ladder.

Step 1: the goal in measurable terms. "Increase activation of new users from 28% to 35% this quarter" (C05).

Step 2: what would make an option good for that goal? Ask: who must be affected (new users, not all users), by how much on the goal metric (impact on activation), how sure are we, how much capacity does it take, what else does it affect (risks, dependencies), and does it open later options? These become the criteria.

Step 3: weights and a flip test. Weights say which criteria matter more *for this goal*. Test them: change the weights modestly and see whether the ranking changes. If the top option stays on top across reasonable weights, the choice is robust; if it flips, you have found the tradeoff that matters and should discuss it explicitly.

Example (invented). The goal is to shorten the time for new sellers on a marketplace to make their first sale. Criteria: impact on time to first sale (weight 40%), reach among new sellers (25%), confidence (15%), ease (20%). A guided listing wizard scores 4.45 out of 5, pricing suggestions 3.30, bulk upload 2.85 and an analytics dashboard 2.35. With the weights shifted to favor ease (impact 25%, reach 25%, confidence 15%, ease 35%), the wizard is still first at 4.15, and bulk upload (3.15) and pricing suggestions (3.00) swap places. The top choice is robust; the second place is not, and that is worth saying.

Why it works. Criteria from the goal keep the ranking about what the team is trying to do.

When not to build a scorecard. For two options, a short written comparison against the goal is clearer than a table.

C15.4 Tradeoff analysis: speed, quality, scope, and short term against long term

Every plan trades among scope (what is delivered), time (when), quality (how well) and resources (who and how many). FW-C15-05 is the triangle. Changing one moves the others, and the useful question is *which is fixed and which can flex?* A date fixed by a launch event and a quality bar fixed by safety leave scope as the lever; a scope fixed by a contract leaves time or resources.

A **time-box** is a fixed period given to a piece of work, with scope that flexes to fit. It turns "how long will this take?" into "what is the best we can do in two weeks?", which protects the date and forces choices about scope. **Scope creep** is the growth of a project's scope after it starts, without a matching change in time, resources or other scope, which delays or degrades delivery.

Adding people. A common response to a tighter deadline is to add engineers. Example (invented): three engineers have 8 weeks, a capacity of 24 person-weeks, for 10 stories at about 2.4 person-weeks each. The deadline moves to 4 weeks. Capacity is 12 person-weeks, enough for 5 stories. Add two engineers: each is about 25% productive in week 1, 50% in week 2 and fully productive after, so each delivers $0.25 + 0.5 + 1 + 1 = 2.75$ person-weeks in four weeks, 5.5 for both.

But the three existing engineers lose about 20% of two weeks to onboarding: $3 \times 2 \times 0.2 = 1.2$ person-weeks. Net gain 4.3, for 16.3 person-weeks, about 6.8 stories. Adding two people gets roughly seven stories, not ten, and the number of person-to-person communication paths grows from 3 (three people) to 10 (five people: $5 \times 4 \div 2$), which raises overhead. This is the point of Brooks's law (adding people to a late project tends to make it later [V: Brooks, The Mythical Man-Month (1975)]): people are not interchangeable with time.

Quality and the long term. Cutting quality (skipping tests, rough edges) is a loan: it saves time now and costs more later, with interest in defects, slower changes and user trust. Say which quality you are cutting, why it is safe, and when you will repay it. Accessibility, safety

and honest information are not cuttable (C10). Likewise, short-term gains (a feature that lifts a number this quarter) can cost the long term (complexity, a dark pattern, a precedent). Reversibility helps: prefer the option you can undo (C02).

Why it works. Naming what is fixed and what flexes turns a vague squeeze into a choice.

When not to over-model. In an interview, name the fixed point, the lever you would pull and what it costs; no full model is needed.

C15.5 Opportunity sizing for prioritization

To compare options you need a rough size for each, and you need it to carry the uncertainty. C08.6 gave a sizing ladder (reach $\times$ baseline $\times$ lift $\times$ value, with haircuts); C04 gave estimation habits. For prioritization, add two things.

Expected value. Multiply the size of the prize by the probability it materializes. Option A: a \$2,000,000 annual benefit with a 60% chance of working, expected value \$1,200,000. Option B: \$5,000,000 with a 20% chance, \$1,000,000. B has the bigger prize and the smaller expected value. Divide by effort for value per unit of capacity: if A takes 4 person-months and B takes 10, A returns \$300,000 per person-month and B \$100,000. The decision is sensitive to the probabilities, so state them as assumptions, and look for tests that would sharpen them (C15.6).

Bias to correct. Estimates of effort tend to be low and estimates of benefit high (the planning fallacy [V: Kahneman and Tversky (1979), as summarized in reference-class forecasting sources]). A defense is a reference class: how long did similar projects take, and what benefit did similar features deliver? Use it as a sanity check on your chain ([sanity-check]), C04).

Ranges, not points. Give a low-base-high range for each option and see whether the ranking survives the ranges. If option A's low case beats option B's base case, the choice is easy; if the ranges overlap, the choice depends on learning.

Why it works. Expected value per unit effort puts unlike options on one scale and makes the probability visible. **When not to rely on it.** When the options serve different goals (safety against growth), the unit does not convert; use criteria and constraints instead (IQ-C15-05).

C15.6 Assumption mapping, risk-based sequencing and the minimum viable product

An **assumption** is a belief a plan depends on that has not yet been verified. Plans fail because an assumption was false and no one tested it. FW-C15-06 is the map.

List and place. List the assumptions a plan rests on (users want it, they will pay, the technology works, we can reach them, the law allows it, partners will cooperate). Place each on two axes: *how much the plan depends on it* (importance) and *how little evidence you have* (uncertainty). The top-right corner (important and uncertain) holds the *riskiest assumptions*.

Sequence by risk. Test the riskiest assumption first, with the cheapest test that could prove it wrong. That is "risk-based sequencing": do the work that retires the most uncertainty per unit of effort before the work that is merely certain. It is the reason a team should not spend six months building polish on a feature whose central assumption (anyone wants it) is untested.

The minimum viable product. An **MVP** (minimum viable product) is the smallest thing that can test the riskiest assumption with real users. It is defined by the question it answers, not by being small or rough. Forms include: a *concierge* test (a person performs the service by hand), a *wizard-of-oz* test (the interface looks real, people operate it behind the scenes), a *fake door* (a page or button for a feature that does not exist yet, to measure demand), a *prototype* (a clickable mockup for usability), and a *single-feature release* (build one slice). Choose the form that matches the assumption: demand needs a fake door or a concierge test; usability needs a prototype; feasibility needs a technical spike.

Success and kill criteria in advance. Before the test, write what result means "go," "stop" and "extend," as in the experiment decision rule (C14.5). Then you cannot move the goalposts afterward. Link to the riskiest-assumption test in C06 (FW-C06-07) for the discovery version.

Why it works. It converts a big bet into a sequence of cheap questions. **When not to test first.** When the cost of the full build is small, the change is reversible, or the assumption is already well evidenced.

C15.7 Saying no and negotiating scope

Prioritization means refusing good ideas, and refusing well is a skill. FW-C15-07 is a script with four parts.

1. **Acknowledge the goal or the need** behind the request ("You need X because customers are asking for Y").
2. **Show the tradeoff, not just the no** ("To build this we'd need to drop Z, which affects W").
3. **Offer an alternative** (a smaller version, a later date, a different route that serves the need).
4. **Say what would change the answer** ("If we learn it affects more customers, or capacity opens, we'll revisit; let's check at the end of the quarter").

Scope negotiation. When scope grows, ask the trade question: "If we add this, what comes out, or what moves?" Writing the roadmap as a table of what is in, what is out, and what is later makes the tradeoff visible and the conversation about priorities, not personalities. For stakeholders with authority, bring the options and your recommendation, with the cost of each, and let them decide with eyes open; do not hide the tradeoff to avoid conflict. C32 develops influence without authority.

Why it works. A reasoned, kind no keeps the relationship and the credibility of the roadmap. **When not to say no.** When the request is cheap, important and consistent with the goal; then yes is a decision too.

C15.8 Interview forms: which feature first; what to cut

Section 4 sets out the answer architecture and the types: which feature first, what to cut when the deadline moves, stakeholder conflict, two teams wanting the same engineers, scoping an MVP, and choosing between a small sure thing and a big uncertain one.

C15.9 Worked examples

Section 5 gives five worked examples: a scoring that flips when anchored to the goal, a weighted scorecard with a flip test for a marketplace, a deadline squeeze, an assumption map with an MVP test and a customer request that would displace the roadmap. Section 11 adds eight questions with model answers, including an Apple question on three candidates for a system release, and a Meta question and a Microsoft question.

3. Frameworks

Eight frameworks, set out below. The tradeoff table (FW-C02-03), the decision classifier (FW-C02-04), the pre-mortem (FW-C02-05), the decision memo (FW-C02-06), the riskiest-assumption test (FW-C06-07), the impact sizing ladder (FW-C08-06), the goal ladder (FW-C07-02) and the experiment decision-rule table (FW-C14-05) are reused, not repeated.

FW-C15-01 · The prioritization answer spine · Original

- **Inputs:** a set of options, a capacity or deadline, and 20 to 25 minutes.
- **Outputs:** an answer with seven visible steps: goal and constraints, criteria, options and sizing, score and sensitivity, tradeoffs and risks, decision, what we won't do.
- **Assumptions:** the interviewer will answer clarifying questions about the goal and constraints.
- **Tradeoffs:** structure makes the reasoning scorable and can crowd out the key insight; adapt to the question.
- **Failure modes:** starting with a framework; criteria not tied to a goal; no decision; no "won't do."
- **Worked example:** Worked example 1, and the model answers in section 11.

Steps: **Goal and constraints** → **Criteria** → **Options and sizing** → **Score and sensitivity** → **Tradeoffs and risks** → **Decision** → **What we won't do, and what would change it.** The time budget is in section 4.

FW-C15-02 · The framework chooser · Original

- **Inputs:** the kind of decision (many similar items, a release scope, time-sensitive items, strategic bets) and the data available.
- **Outputs:** a chosen framework with the reason, its main failure mode and a check that offsets it.
- **Assumptions:** the data needed by the framework exists or can be estimated.
- **Tradeoffs:** a framework gives structure at the cost of simplification.

- **Failure modes:** one framework for everything; scoring with no data; using a score as the decision.
- **Worked example:** the table in C15.2 and IQ-C15-01.

Pairings: **many similar items with data → RICE; fast triage → ICE; release scope for a fixed date → MoSCoW; unlike criteria → weighted scoring; time-sensitive items → cost of delay; customer expectations → Kano.** Always add: source of each number, sensitivity, fit with the goal.

FW-C15-03 · RICE · Established

- **Origin:** popularized by the product team at Intercom, described in a company blog post (the post is dated 5 January 2018; accounts of when RICE was first used range from 2015 to 2017) [V: Intercom blog, "RICE: Simple prioritization for product managers" by Sean McBride; dated 5 January 2018 per the site and a content aggregator; the year RICE was first used is not established].
- **Inputs:** for each option: reach (people or events per period), impact (a scale such as 0.25, 0.5, 1, 2, 3), confidence (a percentage) and effort (person-months).
- **Outputs:** a score = reach × impact × confidence ÷ effort, and a ranking.
- **Assumptions:** reach and impact are measured against the same goal; effort is comparable across options.
- **Tradeoffs:** quick, comparable and transparent; easy to game and to mistake for precision.
- **Failure modes:** reach not tied to the goal; impact as a guess; confidence inflated for favorites; effort underestimated.
- **Worked example:** Worked example 1 (same options, two goals).

Formula: **score = (reach × impact × confidence) ÷ effort.** Add: write the goal first, define reach as people on whom the goal depends, and rerun with the least certain number changed.

FW-C15-04 · The goal-to-criteria ladder · Original

- **Inputs:** a goal and a list of options.
- **Outputs:** measurable criteria derived from the goal, weights with reasons, a scored table and a flip test.
- **Assumptions:** the goal is agreed; weights can be argued for.
- **Tradeoffs:** weighting forces explicit values and can look more objective than it is.
- **Failure modes:** generic criteria; weights tuned to a preferred option; no flip test.
- **Worked example:** Worked example 2, and the example in C15.3.

Steps: **Goal (measurable) → What makes an option good for it → Criteria and weights → Score 1 to 5 → Flip test (change the weights) → State what is robust and what is not.**

FW-C15-05 · The tradeoff triangle and dials · Original

- **Inputs:** a plan with scope, time, quality and resources, and a change (a tighter date, new scope).
- **Outputs:** the fixed point, the lever chosen, its cost, and a plan to repay any quality borrowed.
- **Assumptions:** at least one of the four can flex.
- **Tradeoffs:** every lever has a cost; the choice is about which cost is acceptable.
- **Failure modes:** adding people late; cutting quality silently; "everything is fixed."
- **Worked example:** Worked example 3.

Lever	Typical cost
Cut scope	Features missing; may break the value proposition if the wrong cut
Move the date	Delay; dependencies; commitments
Add resources	Onboarding and coordination cost; works best early, for separable work
Lower quality	Defects, rework, trust; never for safety, accessibility or honesty

FW-C15-06 · The assumption map · Original

- **Inputs:** a plan and its dependencies.
- **Outputs:** a list of assumptions placed by importance and evidence, the riskiest identified, a test or MVP for it with success and kill criteria, and the order of work.
- **Assumptions:** assumptions can be found by asking "what must be true?"
- **Tradeoffs:** testing delays the build and reduces the chance of building the wrong thing.
- **Failure modes:** listing assumptions without ranking; an MVP that tests nothing; criteria set after the result.
- **Worked example:** Worked example 4.

Assumption	Importance (H/M/L)	Evidence (strong/weak/none)	Test	Success / kill criteria
------------	--------------------	-----------------------------	------	-------------------------

Rule: test the high-importance, weak-evidence assumption first, with the cheapest test that could prove it wrong.

FW-C15-07 · The scope-negotiation script · Original

- **Inputs:** a request that does not fit the plan.
- **Outputs:** a four-part response: acknowledge the need, show the tradeoff, offer an alternative, name what would change the answer.
- **Assumptions:** the request has a legitimate need behind it.
- **Tradeoffs:** a reasoned refusal takes longer than yes or no and keeps trust.
- **Failure modes:** a bare no; a hidden tradeoff; a yes that silently breaks the plan.
- **Worked example:** Worked example 5, and IQ-C15-07.

Script: "I see you need [need]. To do it we'd have to [drop/delay X], which means [consequence]. What I can do is [alternative]. If [condition], I'll revisit."

FW-C15-08 · The sensitivity check · Original

- **Inputs:** a ranked list and the estimates behind it.
- **Outputs:** the ranking under changed values for the least certain inputs, the break-even value at which the order flips, and a verdict (robust, fragile).
- **Assumptions:** you know which inputs are least certain.
- **Tradeoffs:** extra arithmetic for confidence in the ranking.
- **Failure modes:** varying only the favorite's numbers; no break-even value; ignoring a flip.
- **Worked example:** Worked example 1 and the flip test in Worked example 2.

Steps: **Find the least certain inputs → vary them across a plausible range → note where the ranking flips (the break-even) → say what you'd measure to resolve it.**

4. Answer architecture

The prioritization answer, for a 25-minute round

Step	Minutes	What to produce
Goal and constraints	3	The goal in measurable terms; capacity, deadline, dependencies; what is fixed
Criteria	4	Criteria derived from the goal, with weights and reasons
Options and sizing	6	Each option sized with a chain and a range; expected value if useful
Score and sensitivity	4	A scored table; what happens when the least certain number changes
Tradeoffs and risks	3	What each choice gives up (opportunity cost); the riskiest assumption and its test
Decision	3	The choice, the sequence, what we won't do, what would change it
Slack	2	

Pacing rules. If the interviewer gives the goal, spend the saved time on sizing and sensitivity. If you run behind, keep the goal, the criteria, a decision and the "won't do." Treat questions as steering.

How to say it. Lead with the goal: "I'll rank these against [goal]; my criteria are [two or three]." State the least certain number and what flips: "This ranking holds unless [X] is below [value]."

How to close. Two sentences: "I'd build [option] first because it scores highest on [criteria] for [goal], and I'd defer [options] because [reason]. The riskiest assumption is [assumption], which I'd test with [cheap test]; I'd change the order if [condition]."

Type: which feature first

Use the spine and the goal-to-criteria ladder. Size options, run sensitivity, decide.

Type: what to cut when the deadline moves up

Use the tradeoff triangle: name the fixed point and the lever, cut scope by the goal (what is essential to the core value), list what you defer, and say what you protect (quality, accessibility).

Type: stakeholder or customer conflict

Use the scope-negotiation script; bring options with costs; separate the need from the requested solution.

Type: two teams want the same engineers

State the criteria that apply across both (contribution to company goals, cost of delay, risk), treat non-negotiable constraints (safety, legal) as constraints and not as items to trade, and propose an allocation with a review date.

Type: scope an MVP

Name the riskiest assumption, choose the MVP form that tests it, and write success and kill criteria first.

Type: the short version (10 minutes)

Goal, criteria, a quick sized comparison, the decision and one risk. Protect the goal and the decision.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"What's the goal?"	You started with a framework	State the goal and the constraint first
"Where did that number come from?"	False precision	Give the chain and a range; label as assumption
"What if confidence is lower?"	Sensitivity	Rerun and show whether the ranking flips
"What are you giving up?"	Opportunity cost	Name the best alternative and its value
"Why not do both?"	Capacity	State the constraint and what doing both would cost
"The VP wants the other one."	Stakeholder conflict	Use the script: need, tradeoff, alternative, trigger
"Could you test first?"	Risk-based sequencing	Name the riskiest assumption and the cheapest test

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption. The arithmetic was computed. Each follows the prioritization spine (FW-C15-01) and is written as notes you could say aloud.

Worked example 1 — The same five features, two goals: a ranking that flips (Illustrative)

The situation. A consumer app has five candidate features and capacity for one or two in the quarter. A team lead scores them with RICE (FW-C15-03) and the dark-mode feature comes out on top. The company's goal for the quarter, set by leadership, is to raise new-user activation.

The first scoring (reach counts all users). Reach is people per quarter, impact on a 0.25 to 3 scale, confidence a percentage, effort in person-months (invented).

Feature	Reach	Impact	Confidence	Effort	Score
B Dark mode	100,000	0.5	90%	1	45,000
E Onboarding checklist	30,000	2	80%	1.5	32,000
A Saved searches	40,000	1	80%	2	16,000
D Faster checkout	60,000	1	70%	3	14,000
C Referral program	20,000	2	50%	4	5,000

Dark mode wins because its reach is the whole user base. But the goal is activation of *new* users.

Rescored against the goal. Reach becomes the new users who would encounter the feature (about 30,000 new users a quarter, fewer for features only some use); impact becomes the effect on activation specifically.

Feature	Reach (new users)	Impact on activation	Confidence	Effort	Score
E Onboarding checklist	30,000	3	80%	1.5	48,000
D Faster checkout	30,000	1	70%	3	7,000
B Dark mode	30,000	0.25	90%	1	6,750
A Saved searches	8,000	0.5	80%	2	1,600
C Referral program	6,000	1	50%	4	750

Dark mode drops from first to third, and the onboarding checklist, second before, leaps to first by a wide margin. Nothing about the features changed; the goal did.

Sensitivity (FW-C15-08). The checklist's confidence is the least certain number (it is a new idea). At 50% instead of 80%, its score is $30,000 \times 3 \times 0.5 \div 1.5 = 30,000$, still more than four times the runner-up. Its effort would have to exceed about 10 person-months ($30,000 \times 3 \times 0.8 \div 7,000 \approx 10.3$) before it lost first place. The ranking is robust; the order of D and B, 7,000 against 6,750, is not, and it would be a poor use of time to argue about it.

Decision. Build the onboarding checklist. The second slot is open: D and B are close, and neither is compelling for this goal. I'd use the second slot to test the riskiest assumption in the checklist (that new users will complete it) with a prototype, not to commit to D or B. What I will not do this quarter: dark mode and the referral program, with the reason stated (not tied to activation, or low confidence and high effort) and a date to revisit.

What this shows. The framework's output depended on the definition of reach and impact, which come from the goal. A ranking is only as good as the goal behind it, and the sensitivity check shows which parts are worth debating.

Worked example 2 — Criteria from the goal: seller tools in a marketplace (Illustrative)

Setting: a seller tools team in an online marketplace of the kind Amazon operates. No fact about any real company is claimed.

The goal. Shorten the time it takes a new seller to make a first sale (a median of 21 days, invented). The scores below are my judgments (invented).

Criteria and weights (FW-C15-04). Impact on time to first sale, 40% (it is the goal); reach among new sellers, 25% (the goal concerns them); confidence, 15% (evidence for the impact); ease, 20% (capacity is limited). Each option is scored 1 to 5.

Option	Impact	Reach	Confidence	Ease	Weighted score
Guided listing wizard	5	5	4	3	4.45
Pricing suggestions	4	4	2	2	3.30
Bulk upload	2	2	5	4	2.85
Analytics dashboard	1	3	4	3	2.35

(Example: wizard = $0.4 \times 5 + 0.25 \times 5 + 0.15 \times 4 + 0.2 \times 3 = 2.0 + 1.25 + 0.6 + 0.6 = 4.45$.)

Flip test. Shift the weights toward ease (impact 25%, reach 25%, confidence 15%, ease 35%): wizard 4.15, bulk upload 3.15, pricing suggestions 3.00, dashboard 2.65. And toward impact (50%, 30%, 10%, 10%): wizard 4.70, pricing suggestions 3.60, bulk upload 2.50, dashboard 2.10. The wizard is first under all three. Second place changes: pricing suggestions under impact-heavy weights, bulk upload under ease-heavy weights. So the choice of the wizard is robust and the second choice depends on how much we value ease.

Decision and the riskiest assumption. Build the wizard first. Pricing suggestions have the lowest confidence (2): the assumption that sellers will trust and use suggested prices is untested. Before building, run a cheap test: show suggested prices to a sample of new sellers in a concierge format and measure acceptance (C15.6). If acceptance is high, it rises to second.

What I would say to the team. "We're ranking against time to first sale, and the wizard wins however we weigh ease against impact. The dashboard is useful for established sellers and does little for this goal, so it's not now. We'd revisit it when our goal shifts to retention of established sellers."

What this shows. Criteria derived from the goal made the tradeoff explicit, and the flip test showed which part of the ranking to trust.

Worked example 3 — The deadline moves up: what do you cut? (Illustrative)

The situation. A team of three engineers has 8 weeks to build a feature with 10 stories (invented). Leadership moves the date to 4 weeks. The PM must respond.

The arithmetic (FW-C15-05). Capacity at 8 weeks: $3 \times 8 = 24$ person-weeks, about 2.4 per story. At 4 weeks: 12 person-weeks, enough for 5 stories.

Option A: add two engineers. Each new engineer is about 25% productive in week 1, 50% in week 2 and fully productive after, so over 4 weeks each delivers $0.25 + 0.5 + 1 + 1 = 2.75$ person-weeks; for two, 5.5. The three existing engineers lose about 20% of two weeks to onboarding: $3 \times 2 \times 0.2 = 1.2$. Net gain 4.3, for a capacity of 16.3 person-weeks, or about 6.8 stories. Communication paths rise from 3 to 10. Adding people gets about seven stories, not ten, and costs two engineers elsewhere.

Option B: lower quality. Skip testing and review to save, say, 25% of effort: capacity of 12 becomes an effective 16, or about 6.7 stories, and the defects arrive later with interest. Not acceptable for the core path or for accessibility and security, and it should be a declared loan with a date to repay.

Option C: cut scope. Keep the 5 to 6 stories that deliver the core value, and defer the rest. The question is which. The goal is the guide: the stories without which the feature cannot deliver the user's main job stay; polish and edge-case stories go. Assume 6 of the 10 are core.

Option D: move the date a week. Five weeks gives 15 person-weeks, 6.25 stories, which may be enough for the 6 core stories. Worth asking whether the 4-week date is a hard constraint (a launch event, a contract) or a preference.

Recommendation. First ask what is fixed: if the date is fixed, cut scope to the 6 core stories, add at most one engineer for separable work that can start on day one, protect quality on the core path, and ship the deferred stories in a follow-up about three to four weeks later with a date (four stories at 2.4 person-weeks each is 9.6 person-weeks, about three weeks for three engineers, plus review). If the date has some give, take one more week and keep scope. Present both to leadership with the cost of each: "In four weeks we can

deliver six of ten stories, the core flow, and the rest about four weeks later; or five weeks gets the same six with less risk. Adding people gets us to about seven stories at the cost of two engineers' other work. I'd choose scope."

What I would not do. Silently compress everything and hope; add many people to a short, interdependent project; or cut tests on the payment path.

What this shows. The arithmetic made it clear that people and quality are not free substitutes for time, so the lever is scope, chosen by the goal.

Worked example 4 — Assumption map and an MVP test: group savings (Illustrative)

The idea. A savings app wants to add "group savings goals," where friends save toward a shared goal. Engineering estimates four months to build (invented).

The assumption map (FW-C15-06).

Assumption	Importance	Evidence	Test
People want to save toward goals with friends	High	Weak (a few interviews)	Interviews; concierge test
Invited friends will join	High	None	Concierge test with a join-rate threshold
We can hold pooled money within the rules	High	None	Legal review (cheap, early)
Group goals improve retention	Medium	None	Later, after launch
Engineering can build it in four months	Medium	Moderate	Technical spike

The riskiest assumption. That invited friends will join. If they do not, there is no group, and nothing else matters. It is high-importance and has no evidence. The legal question is also high-risk and cheap to test, so start it in parallel.

The MVP: a concierge test. Create 30 groups by hand (a person sets each up and invites friends by message), with 4 invitees per group: 120 invitees. No new software. It takes about two weeks of one PM and one operations person, against four months of engineering.

Success and kill criteria, written first (C14.5). Success: at least 40% of invitees join within 7 days (48 of 120). Kill: 25% or fewer (30 or fewer). Between: extend or redesign. How sure would the results be? If 48 of 120 join, the 95% interval for the true join rate is about 32% to 49%; if 30 of 120, about 18% to 33%; if 36 of 120 (30%), about 23% to 39%, which would straddle the bars and call for more data or a redesign of the invitation. The sample is small, and it answers the question of whether there is any signal, not the final rate.

Sequence. (1) Legal review and the concierge test now, in parallel; (2) if the test passes, a technical spike and a thin build of the smallest slice (create a group, invite, contribute); (3) retention comes later, after real use.

What this shows. The test cost two weeks instead of four months, and the criteria were set before the data, so the result could actually change the decision.

Worked example 5 — A customer request that would displace the roadmap (Illustrative)

The situation. A business-software company has \$50,000,000 in annual revenue (invented). Its largest customer, 8% of revenue (\$4,000,000), asks for a custom feature and hints that it may leave if it does not get it. Engineering estimates 9 person-months, about \$225,000 at \$25,000 a person-month. Building it would displace a planned item with an expected value of \$1,200,000 a year (a 60% chance of \$2,000,000). Upkeep would be about 20% of the build cost a year, \$45,000.

The arithmetic. If the customer leaves with a 25% probability when refused, the expected loss is $0.25 \times \$4,000,000 = \$1,000,000$. Building the custom feature protects that \$1,000,000 but displaces \$1,200,000 and adds \$45,000 a year of upkeep: a net of about –\$245,000 a year. The break-even is the churn probability at which protection equals cost: $(\$1,200,000 + \$45,000) \div \$4,000,000 = 31\%$. If the customer's real chance of leaving is above 31%, the custom feature pays; below, it does not. The estimate of the probability is the key assumption, and the cheapest way to improve it is to talk to the customer.

The approach (FW-C15-07). Do not decide on the number alone. Understand the need behind the request: what is the customer trying to accomplish, and does a general feature or a configuration serve it? If other customers have the same need, a general version has a larger payoff and spreads the cost. Then respond with the script: acknowledge the need ("this matters for your operations"), show the tradeoff ("a custom build would delay the reporting upgrade that 40 other customers are waiting for"), offer an alternative (a configurable version in the next quarter, and an interim workaround now), and name the trigger ("if the workaround doesn't hold, let's meet again in six weeks with your numbers").

Precedent and concentration. Say yes to a one-off for the largest customer and the next one will ask. And revenue concentration of 8% is a risk to manage in its own right, not just a reason to comply.

Decision. Do not build the custom feature as specified. Offer the general version on a date, an interim workaround, and an executive-level conversation. Record the 31% break-even and revisit it if the customer's signals change.

What this shows. The expected-value arithmetic and its break-even gave a defensible no, the script made it constructive, and the need behind the request opened better options.

6. Baseline, Strong, Standout

Prompt: "You have five features and capacity for two. Which do you build?"

Baseline answer

"I'd use RICE: I'd estimate reach, impact, confidence and effort for each and build the two with the highest scores."

What is wrong: a framework named, nothing about the goal or the criteria. The scores will come from nowhere, and the answer does not say what is given up or what would change the ranking.

Strong answer

"First I'd ask what we're trying to achieve this quarter and what is fixed. Suppose the goal is to raise new-user activation, and capacity is two features. I'd derive the criteria from that goal: effect on activation among new users, how many new users it reaches, how sure we are, and effort.

"I'd size each option with a reason: reach as the new users who encounter it, impact as the expected change in activation, confidence from evidence such as past tests or interviews. I'd rank them, then change the least certain number to see if the order holds. I'd pick the top two, say why, and say what I'd defer and why. I'd name the riskiest assumption in the top choice and how I'd test it cheaply."

Standout answer

Everything in the Strong answer, plus:

"I'd keep the tradeoffs explicit: building A and B means not building C, whose value I'd estimate at X, so the opportunity cost is X. I'd show the sensitivity: the top option stays first unless its effort is more than about three times my estimate, which is unlikely, while the second slot is a near tie, so I'd use it to test an assumption and not commit.

"I'd check for second-order effects: one of the options changes the sign-up flow and could disturb existing users, so I'd ask how it affects them (the legacy-user question) and put a guardrail on it. I'd write kill criteria for the test before running it.

"I'd also say what I will not do, with a date to revisit, so stakeholders know the 'no' is a 'not now' with a reason. And I'd be ready to change the order if the goal changed: if the quarter's goal moved to retention of existing users, two of the deferred options would rise.

"Last, I'd treat a score as a way to organize an argument and not as the argument. What would change my mind: evidence that my reach estimate for the top option is off by more than a factor of three, or a dependency that blocks it."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
States the goal and derives criteria from it	Names the opportunity cost and the sensitivity of the ranking
Sizes options with reasons and picks with a reason	Tests the riskiest assumption cheaply, with kill criteria set first

From Baseline to Strong	From Strong to Standout
Names what is deferred	Adds a guardrail for second-order effects and a revisit date

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Apple [as of 2026-10]. Apple ships major system software updates on a regular cadence and designs hardware, software and services together Public [V: Apple: WWDC in June, public releases in September each year (iOS 26 on 15 September 2025); Apple results release describes six software platforms across all devices and its services]. Third-party guides report that prioritization prompts at Apple involve choosing among candidate features for a release, and reward a clear goal, a small number of well-made things and attention to privacy and consistency across devices Inferred. Practical consequence: derive criteria from a user goal, say what you would not ship, and mention effects on other devices and on developers.

Company lens: Google [as of 2026-10]. In 2011 Larry Page told investors the company would put "more wood behind fewer arrows," in a year when Google closed products including Google Health and PowerMeter (announced in June) and Labs (announced in July) Public [V: CS-C15-02, Google's 20 July 2011 post and Fortune] (section 10). Google has also written publicly about using objectives and key results Public [V: Doerr, Measure What Matters (2018); Google re:Work OKR guidance]. Candidates commonly report prioritization prompts at large scale Inferred. Practical consequence: show that you can cut as well as add, and tie priorities to a measurable objective.

Company lens: Amazon [as of 2026-10]. Amazon's leadership principles include Bias for Action and Frugality, and its founder has described decisions as one-way or two-way doors, with speed encouraged on the reversible ones Public [V: Amazon's Leadership Principles; Bezos 2015 shareholder letter]. Candidates commonly report prompts that probe the customer's problem first Inferred. Practical consequence: classify the decision's reversibility, move fast on two-way doors and say how you would work backwards from the customer.

Company lens: Meta [as of 2026-10]. Candidates commonly report prioritization prompts where growth goals and safety or integrity goals compete for the same engineers Inferred [unverified: candidate-reported]. Practical consequence: separate constraints (safety minimums) from tradeable goals, and propose an allocation with guardrails and a review date.

Company lens: Microsoft [as of 2026-10]. Microsoft's products serve large enterprise customers with long-lived commitments and different needs Inferred [unverified: author's reasoning from public product descriptions; no source checked]. Practical consequence: separate the needs of different customers and the buyer from the user, weigh strategic customers without letting one dictate the roadmap, and prefer general solutions to custom ones.

Company lens: Netflix [as of 2026-10]. Netflix's published culture material describes informed decision-making by accountable individuals, with context over control Public [V: Netflix culture memo, June 2024]. Candidates commonly report data-informed prompts Inferred. Practical consequence: say who decides and on what evidence, and what you would test before committing.

Company lens: Stripe [as of 2026-10]. Stripe serves developers who build on its products and documents API versioning to keep integrations stable Public [V: Stripe API versioning documentation and engineering blog]; candidates report developer-product prioritization prompts Inferred. Practical consequence: weigh the cost to developers of changes (breaking changes are expensive) alongside new capability.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Starting with a framework	You say "RICE" before any goal	State the goal and constraints first
2	Criteria not from the goal	Generic "impact and effort"	Derive criteria from the goal and weight them
3	Reach not tied to the goal	Reach is all users for a new-user goal	Define reach as the people the goal concerns
4	False precision	Scores to many digits	Give ranges and a sensitivity check
5	No opportunity cost	Only the benefits of the chosen option	Name the best alternative given up
6	Pros and cons with no decision	"It depends"	Decide, and say what would change it

#	Mistake	How to detect it	Fix
7	Building before testing the riskiest assumption	Six months to find out nobody wants it	Map assumptions; test the riskiest cheaply
8	Adding people to a late project	"We'll add engineers"	Do the arithmetic; cut scope
9	A bare no	Refusal with no reason or alternative	Use the four-part script
10	Cutting quality silently	Skipping tests to hit a date	Declare the loan; protect core, safety and accessibility

9. Mental models applied

Four models, each tied to a decision in this chapter. [\[\[reversible-decision\]\]](#) is from C02 and [\[\[pareto-principle\]\]](#) from C03.

Opportunity cost (MM-opportunity-cost, see [\[\[prioritization\]\]](#)). Every yes is a no to something else. The real price of a choice is the value of the best alternative forgone, so ask not "is this worth doing?" but "is this worth more than the best other use of the same capacity?" It is the reason the displaced item was part of the custom-feature arithmetic in Worked example 5.

The outside view (MM-outside-view). Estimate from how similar projects actually went, not from the story of this one. Teams tend to predict their own project will go better than the typical one [V: Kahneman and Tversky (1979), as summarized in [reference-class forecasting sources](#)]. Ask: how long did the last three comparable projects take, and what benefit did they deliver? Use the answer to adjust effort up and benefit down.

The vital few (MM-vital-few, see [\[\[pareto-principle\]\]](#), C03). A small share of items usually accounts for most of the value, which is why the funded half of a long list can deliver most of the benefit (CS-C15-01). Look for the items with the highest value per unit effort and the largest absolute value, and be willing to leave the long tail.

Reversibility (MM-reversibility, see [\[\[reversible-decision\]\]](#), C02). Move fast on choices you can undo; slow down on those you cannot. This changes how much evidence to demand: a two-week test justifies a reversible launch, while an irreversible commitment deserves the full analysis, including a pre-mortem.

Engineering, design and data-science perspectives

- **Engineering.** Estimates are ranges, not points; ask for a range and what would change it. Ask which options have hidden dependencies or technical risk, and which could be split into a thin first slice. Engineers often see the cheaper route to the same goal.
- **Design and research.** Ask what the research says about the riskiest assumption and whether a prototype test can answer it in days. Designers can often scope a smaller version that delivers most of the experience.

- **Data science.** Ask for the baseline and the reach numbers you are using, and for help sizing impact with a range. Agree the success and kill criteria for any test before it runs (C14). Ask what the data cannot tell you.
- **Sales, support and stakeholders.** They hold information on what customers ask for and what costs the business. Bring them the criteria and the tradeoffs early; people accept a "not now" they helped shape better than one handed down.

Checklist: before you commit to a priority order

1. Is the goal stated in measurable terms, with the constraints?
2. Are the criteria derived from the goal, with weights and reasons?
3. Is each option sized with a chain and a range, and is the source of each number clear?
4. Did I run a sensitivity check and find where the ranking flips?
5. Did I name the opportunity cost and what we are not doing?
6. Did I identify the riskiest assumption and a cheap test, with kill criteria set first?
7. Did I classify reversibility, and protect quality on the core path, safety and accessibility?
8. Do I have the four-part script ready for the stakeholders who will hear no?

10. Case studies

CS-C15-01 · The quarter with twelve priorities · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A product team of six engineers has 30 person-weeks of capacity for the quarter. Stakeholders have submitted twelve items, and the plan calls all twelve "priorities." Last quarter the team shipped three of ten.

Constraints. A fixed capacity, a company goal to raise activation and retention, and a habit of saying yes to every request.

Data. Each item has a value in points (a rough estimate of contribution to the goal, with stakeholders agreeing the scale) and an effort in person-weeks. Total effort is 68 person-weeks, 227% of capacity.

Item	Value	Effort (pw)	Value per pw	Cumulative effort
c Fix sign-up bug	8	2	4.0	2
e Notification settings	6	3	2.0	5
b Search improvements	9	6	1.5	11
a Onboarding revamp	14	10	1.4	21
d Referral v2	7	5	1.4	26 (over capacity)
i Help-center links	4	3	1.3	24 (after skipping d)

Item	Value	Effort (pw)	Value per pw	Cumulative effort
g Export to CSV	5	4	1.25	not funded
f Admin dashboard	6	8	0.75	not funded
j Dark mode	3	6	0.5	not funded
h New integrations	4	10	0.4	not funded
k Profile themes	2	5	0.4	not funded
l Social sharing	2	6	0.33	not funded

Problem definition. As an observable problem: "The plan asks for 68 person-weeks from a team with 30, so the team ships a few things late and the rest never."

Analysis. Hold back a 20% buffer for bugs and surprises: a planning capacity of 24 person-weeks. Rank by value per person-week and fund down the list until capacity is used: c (2), e (3), b (6), a (10) is 21. The next, d, would reach 26, over capacity; the item with the next-best ratio that fits is i (3), reaching 24. Referral v2 (d) ties the onboarding revamp (a) on ratio at 1.4, but a has twice the absolute value (14 against 7), so it is funded first. The funded items are c, e, b, a and i: 24 person-weeks, 35% of the requested effort, delivering $8 + 6 + 9 + 14 + 4 = 41$ of the 70 points of value, 59% of the total (the vital few, C15 mental models). The scores are rough, so sensitivity matters: the top four stay funded under reasonable changes in value; d and i are the marginal pair, and swapping them changes little.

Options.

Option	Optimizes	Costs
A. Keep all twelve as "priorities"	No one is told no	Most will not ship; those that do ship late; the team's credibility
B. Fund five items to capacity with a buffer; list the rest as not now with a revisit date	Value per unit capacity and predictability	Seven stakeholders are told no or later
C. Ask for more engineers	Scope	Cost and ramp-up; capacity will still fall short of 68

Tradeoffs. B costs seven conversations and gives a plan that can be met and a clear reason for each decision. A feels easy and fails on delivery. C does not close a gap of this size.

Decision. Choose B. Use the four-part script for each unfunded item (C15.7), share the list of what is funded and what is not, and set a checkpoint at week six to review progress and re-rank. Primary measure: the share of committed items shipped in the quarter (last quarter 30%). Guardrail: quality of shipped items (defects found in the first month). Counter measure: stakeholder satisfaction with the process, since a clear no may cost goodwill.

Outcome. In this fictional scenario the team shipped all five funded items, with the buffer used on two bugs, and revisited the list at week six, adding g (Export to CSV) when a delay freed four person-weeks. Stakeholders of unfunded items were frustrated but could see the criteria; two raised new evidence and had their items re-ranked next quarter (invented).

Alternative outcomes. If the buffer had been too small, the team would have dropped i and told its stakeholders early. If the value scores had been disputed, the team would have run the ranking under each stakeholder's scores and shown that the top four did not change. The lesson: a plan that fits capacity with a buffer, a stated rule for what comes next and a reason for each no beats a list of twelve priorities.

CS-C15-02 · Google, 2011: "more wood behind fewer arrows" · Verified · Advanced

Sources (types): Google's blog post "More wood behind fewer arrows" by Bill Coughran, senior vice president for research and systems infrastructure (Official Google Blog, 20 July 2011), read in full (primary); Fortune's article "Is this what a 'focused' Google looks like?" by Verne Kopytoff (3 July 2012), read in full; and news coverage of Larry Page's remarks on Google's second-quarter 2011 earnings call and of the product closures (TechRadar, NBC Bay Area, Digital Trends, CNN/Wired, CNN Money). The earnings call itself was read through those reports. The analysis is the author's. The sources do not reveal Google's internal criteria or data. Re-read in October 2026.

Situation. Larry Page took over as chief executive of Google in April 2011. On the second-quarter earnings call on 14 July 2011 he said that greater focus had been a big feature of the quarter for him: "more wood behind fewer arrows." In June 2011 Google announced that it would shut down Google Health, an online service for personal health records, and Google PowerMeter, a service for monitoring home energy use. On 20 July 2011, Google announced on its official blog, under the title "More wood behind fewer arrows," that it would wind down Google Labs, the site where it released early prototypes. In its statement, Google said that while it had learned a great deal from launching early prototypes there, greater focus was crucial to make the most of the opportunities ahead; many Labs products would be ended and others folded into other products. Android apps from Labs would remain on Android Market, and an update added to the post the same afternoon said that Google had no plans to change in-product experimentation channels such as Gmail Labs and Maps Labs [V: Google blog post]. A Fortune article of 3 July 2012 reported that Google executives had, in blog posts since the previous year, repeatedly cited focus as the rationale for closing more than 30 products or features; it named Health, PowerMeter, Slide, Buzz and Labs as closed and Knol as due to close that October, and said that Page had reorganized the company into seven groups shortly after retaking the chief executive role [V: Fortune, 3 July 2012].

Constraints. Google had grown a long list of products, and critics had said it had lost focus (Fortune). Engineering and management attention are limited, so each product kept has an opportunity cost in the other things that do not get built. Closing products affects the people who use them, and Labs also served as a public testing ground that some users valued.

Data. As reported: the dates and the products named above; the stated rationale (focus); the continuation of in-product experiments in Gmail and Maps; the reorganization; the count of more than 30 closures reported in 2012. Aaron Levie, chief executive of the online file-storage company Box, told Fortune in July 2012, as Google launched a tablet, a home entertainment device, a computing service and a preview of Internet-connected glasses, that the strategy had become "more wood and more arrows" [V: Fortune, 3 July 2012]: a

reminder that stated intent and later behavior can differ. A community-maintained list of retired Google products and services counted 299, according to a data-visualization page that draws on it and that reported this figure when read in October 2026 [V: secondary].

Problem definition. *The author's reading.* The problem is an allocation problem: too many projects for the capacity, so each gets too little. As an observable problem: "Many products, each with a small team and uncertain traction, spread attention thin and made it unclear which mattered most."

Analysis. *The author's analysis.* Through this chapter's frameworks, the case shows prioritization by *cutting*. The opportunity cost of a product is the best use of the people and attention it consumes (C15.1), and the decision to close Health, PowerMeter and Labs is a statement that other uses are worth more. The stated rule, fewer and bigger bets, is a criterion derived from a goal: focus on areas the company judged central (on the earnings call Page described three groups of products: the core advertising business, high-usage consumer products such as Android, Chrome and YouTube, and new products such as Google+ and Local [V: TechRadar report of the call]; Fortune reports the seven-group reorganization, with executives over advertising, social networking, Android, Chrome, YouTube and local mobile commerce [V: Fortune]). The way it was done shows the script (FW-C15-07) at company scale: give the reason (focus), name what continues (in-product experiments, some Labs products folded in), and offer an alternative where there is one.

The costs are the legacy-user and trust costs from C08: people who relied on a product lose it, and a company known for closing products may find users slower to adopt the next one [inference]. The critic's remark shows the second risk, that a stated priority is only as good as the follow-through. What the sources do not show is the quantitative basis: which products had what traction, what the freed teams built instead, and whether the strategy paid. So the case supports the *method* (cut to fund focus, say why, protect what matters) and not a claim that the specific cuts were right.

Options. *The author's reconstruction of choices a leadership team could have faced; not a record of Google's deliberations.*

Option	Optimizes	Costs
A. Keep all products and add resources	Breadth; no one is told no	Attention stays spread; cost grows; the largest bets are not larger
B. Close low-traction products, redeploy teams to central bets, announce the reasoning	Focus and clarity	Users of closed products; reputational cost
C. Freeze low-traction products in maintenance mode	Avoids closure backlash	Ongoing cost and attention; products linger

Tradeoffs. B recovers capacity and states priorities clearly, at the cost of disappointing users and critics. A is easy and keeps the problem. C is a compromise that leaves much of the cost in place.

Decision. As documented, Google chose B in a series of announcements in June and July 2011 and afterwards, together with a reorganization.

Outcome. Reported: closures of Health, PowerMeter and Labs, and, according to Fortune in July 2012, more than 30 products and features closed with focus cited as the rationale, with the continuation of in-product experimentation [V: Fortune; Google blog post]. Whether it improved Google's results is not established in the sources used here.

Alternative outcomes. *Inference.* Had the closures been announced with no reason, users would have read them as arbitrary, and the focus message would have been lost. Had the freed capacity gone to new scattered projects, the critic's quip would have been true. A reader should note that a famous decision looks clearer in hindsight than it was. The method to carry away: treat what you stop as a prioritization decision with an opportunity-cost rationale, communicate the reason and what continues, protect the people affected where you can, and check later that the freed capacity went where you said it would.

11. Interview questions and model answers

Eight questions: two Beginner, three Intermediate, three Hard. Model answers are **Illustrative** (composite candidates), and every figure is an assumption; the arithmetic was computed. Each is written against the six behaviors in section 1 and uses the prioritization spine (FW-C15-01). Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C15-01 · Beginner · execution · EX

Prompt. How do you prioritize features?

Rubric.

- **Baseline:** "I use RICE" (or another framework), with nothing more.
- **Strong:** Starts from the goal, derives criteria, sizes options and decides.
- **Standout:** Treats frameworks as tools, runs a sensitivity check, names the opportunity cost and the riskiest assumption, and says how stakeholders are brought along.

Model answer. I start with the goal, because the same list ranks differently under different goals. I ask what we are trying to move this quarter and what is fixed: capacity, the date, dependencies. Then I derive criteria from the goal: who must be affected, by how much on the goal metric, how sure we are, and what it costs. If the goal is new-user activation, reach means new users, not everyone.

Then I size each option with a reason and a range, and I use a framework such as RICE only to organize the numbers: reach, impact, confidence, effort. I'd say where each number comes from and what the least certain one is. I rerun the ranking with that number changed; if the top option holds, I trust it, and if two options swap, I know that is where to spend discussion or a small test.

I name the opportunity cost: choosing A means not building B, and I say what B was worth. I look for the riskiest assumption in the winner and the cheapest way to test it before a full build. I decide, say what we are not doing and when we'd revisit it, and I take the reasoning to stakeholders so a "not now" is understood, not imposed.

The mistake I'd avoid is letting a score be the decision. A framework gives a way to talk about the options; the decision is a judgment about the goal, the evidence and the tradeoffs.

Follow-ups.

- **Q.** How do you run a sensitivity check on a ranked list? **A.** "I change the most uncertain inputs and see whether the order moves. If the top item's score depends on a reach estimate I am unsure of, I vary that estimate by a third in each direction and re-rank; if the top two swap, the choice is not robust and I would test the assumption before committing. I also check how much better the first item is than the second: a 5% gap is a coin flip, a 40% gap is a decision. The point is to learn which assumption decides the ranking and to put effort there, not to defend a number."
- **Q.** How do you bring stakeholders along? **A.** "By sharing the goal, the criteria and the capacity before the ranking, so that the result is the consequence of rules they accepted. I show the ranked list with the cut line at capacity, what is below the line and why, and what would change the order. I ask each stakeholder for the evidence behind their item and say what I would need to move it up. People accept a no more easily when they see the tradeoff and have a path to a yes, and a named owner makes the final call."

IQ-C15-02 · Beginner · execution · EX

Prompt. In a scoping meeting, every stakeholder says their items are "must-haves." How do you handle it?

Rubric.

- **Baseline:** "I'd get everyone to agree" or "I'd decide myself."
- **Strong:** Uses a tool such as MoSCoW with a constraint to force tradeoffs.
- **Standout:** Anchors "must" to the goal and the date, uses capacity to test the list, makes tradeoffs visible, and leaves the decision with a named owner.

Model answer. When everything is a must, the word has lost its meaning, so I'd redefine it against a constraint. A "must" is something without which the release fails its goal or cannot ship at the date (a legal requirement, a core flow). I'd restate the goal and the date, then ask of each item: "If this were missing at launch, would the release still deliver its goal?"

Then I'd bring capacity into the room. Suppose the "musts" total 40 person-weeks and we have 24 with a buffer. I'd say: "These can't all be musts; we have capacity for 24. What leaves?" Putting the number in front of people shifts the conversation from defending items to choosing among them. I'd use MoSCoW (must, should, could, won't this time) to sort, with the "won't this time" column given as much attention as "must," because it shows people the item is deferred and not dismissed, with a date.

For disputes I'd offer options with costs rather than argue: "If we keep A, we lose C and D; if we keep C and D, A moves to the next release. Which serves the goal?" The decision belongs to a named owner, usually me with the lead engineer and designer, with the sponsor informed. After the meeting, I'd send the written list with the reasons, so the criteria are on record. What would change my mind: new evidence that a deferred item is a blocker for the goal.

Follow-ups.

- **Q.** How do you use capacity to test the must-have list? **A.** "By adding it up. If the team has 40 person-weeks and the musts total 70, the list is not a list of musts, because at least 30 weeks of it cannot be built. I would put the capacity number on the screen and ask the group to cut or defer until the total fits, starting with items whose absence would not break the goal or the date. That turns an argument about importance into a visible arithmetic problem. Items that stay are the true musts, and the rest become a ranked next list with reasons."
- **Q.** Who decides when stakeholders still disagree? **A.** "A named owner, agreed before the meeting. In most teams the PM proposes and the PM's manager or a product leader decides when there is a cross-team conflict. I would write the options, the tradeoffs and my recommendation in a one-page note, ask the decider for a decision by a date, and record it. Everyone gets heard and the decision is made once. I would also say what evidence would reopen the decision, so that it is not reopened by volume."

IQ-C15-03 · Intermediate · execution · EX · Apple

Prompt. Three candidates for the next system release: better notifications, a new widget, a battery mode. Pick one.

Rubric.

- **Baseline:** "The battery mode, because people care about battery."
- **Strong:** States a goal, derives criteria, compares the three on them, and picks one with a reason.
- **Standout:** Shows the ranking depends on the goal, runs a sensitivity check, names tradeoffs (including effects on developers, privacy and other devices), tests the riskiest assumption, and says what would change the choice.

Model answer. I know the product only from public information; nothing here describes any real plan. I'd first ask what the release is meant to achieve, because the right pick depends on it. Assume the goal is to raise day-to-day satisfaction for the largest group of users across a range of devices, with a secondary aim of keeping developers engaged. I'd say this is my assumption and ask whether it is right.

Criteria from the goal. How many users benefit, how much it improves their day (impact), how sure we are, how much it costs to build, and what it does to others: developers (a widget depends on them), the user's other devices, privacy, and the platform's consistency.

Sizing (invented). Reach as a share of users, impact on a 0.25 to 3 scale, confidence, and effort in person-months.

Option	Reach	Impact	Confidence	Effort	Score
Battery mode	60%	2	70%	4	21.0
Better notifications	80%	1	50%	8	5.0
New widget	40%	1	60%	5	4.8

Reasoning behind the numbers. Battery mode reaches the people who run low in a day (most mobile users sometimes), has a measurable benefit (hours of use) and is a setting, which is lower risk to existing behavior. Notifications touch nearly everyone, but a better version is hard to define, so confidence is low, effort is high, and changes affect developers and the experience of users who like theirs (the legacy-user question). A widget depends on developers adopting it, so its reach is limited by their uptake.

Sensitivity. The battery mode's impact is the key number. If it is 1 instead of 2, its score is 10.5, still more than twice the others. For the notifications to win, their confidence and effort would both have to be far better than assumed. So the choice is robust to this uncertainty. It would change if the goal changed: if the aim were to strengthen the developer ecosystem, the widget would rise, since it creates something for developers to build.

Riskiest assumption and test. That users would turn on a battery mode and accept its tradeoffs (reduced performance or background activity). A cheap test is a prototype setting with a small group measuring opt-in and satisfaction, with a guardrail on complaints about slowness. I'd also check the interaction with other devices (does it sync across a user's devices?).

Decision. "I'd pick the battery mode: it serves the most users with the clearest benefit for the least cost and risk. I'd defer notifications until we can define the improvement, and the widget until developer demand is clearer. What would change my mind: evidence that the benefit is under half what I assumed, or a goal that favors the ecosystem."

Company note. Apple is the setting only: **Inferred** from third-party reports that prioritization prompts at Apple involve choosing among candidate features for a release [unverified: candidate-reported]. No claim is made about any real release, plan or user research.

Follow-ups.

- **Q.** How does the ranking change if the goal changes? **A.** "Each goal rewards different items. If the goal is daily satisfaction for the largest group, better notifications win on reach; if the goal is battery confidence for users of older devices, the battery mode wins on severity for that segment; if the goal is developer engagement, the widget may win because it gives developers a new surface. That is why I ask for the goal first and state my assumption. I would show the interviewer the three rankings in one table so that they can see how the choice depends on the goal."
- **Q.** How would you test the riskiest assumption for your pick? **A.** "Find the assumption that would make the pick wrong, usually that the improvement is noticed. For notifications I would test whether users change behavior after a redesign using a staged release with a holdout, measuring the share of notifications acted on and the share silenced. Privacy constraints shape the test: use on-device measures and aggregated

counts. If the redesign increases silencing, it fails, and I would switch to the next item. A two-week test in a beta group would decide before the full commitment."

IQ-C15-04 · Intermediate · execution · EX · Microsoft

Prompt. Three large enterprise customers each want a different thing: admin controls, better performance, more integrations. They are 12%, 9% and 6% of revenue. How do you prioritize?

Rubric.

- **Baseline:** "Do the biggest customer's request first."
- **Strong:** Separates revenue at risk from breadth of need and effort, and looks beyond the three.
- **Standout:** Distinguishes blockers (compliance) from preferences, finds shared needs, sizes with a framework and sensitivity, protects against concentration and precedent, and plans communication.

Model answer. I'd avoid ranking by customer size, because it makes the roadmap a function of who shouts loudest and ignores the other 197 customers. My approach: separate what each request is *for*, how broad the need is, and what it costs.

Understand the needs. For each customer, ask what outcome they want and what happens if they do not get it. The largest customer's admin controls might be a compliance requirement (a blocker, with a deadline) or a convenience. If it is a compliance blocker, it is a constraint: either we meet it or we risk the account; I'd then look for the smallest set of controls that satisfies the requirement.

Breadth. Check the wider base. Suppose (invented) of 200 customers, 30 have asked for admin controls, 55 for performance and 80 for integrations.

Option	Customers (reach)	Impact	Confidence	Effort (pm)	Score
Integrations	80	1	70%	4	14.0
Performance	55	2	60%	9	7.3
Admin controls	30	1	80%	6	4.0

Integrations score highest because of breadth and low effort. Performance has the highest impact per user but costs the most. Admin controls reach the fewest customers, but that count is not the whole story: the largest customer sits there.

Overlay revenue and risk. The 12% customer is \$X of revenue; if the need is a blocker, the expected loss from refusing (probability of leaving times revenue) may exceed the value of the next item, as in the break-even arithmetic in Worked example 5. I'd talk to them to learn the probability and the minimum viable set.

Decision. Deliver the minimum compliance subset of admin controls first if it is a true blocker (perhaps 2.5 person-months), then integrations, then performance in stages, giving each customer a dated answer. I'd avoid a custom build for any one customer and prefer general features. I'd also address concentration: three customers are 27% of revenue, which is a business risk to reduce by serving the broader base.

Communicate. Share the criteria and the order with all three, with what they get and when, and what the alternative is until then. What would change my mind: learning that the 9% customer's performance issue is causing active evaluation of a competitor.

Company note. Microsoft is the setting only: **Inferred** from candidate reports that enterprise-prioritization prompts at Microsoft separate buyer and user needs **[unverified: candidate-reported]**. No Microsoft data or product behavior is claimed.

Follow-ups.

- **Q.** How do you separate a blocker from a preference? **A.** "By asking what happens if they do not get it. A compliance requirement without which the customer cannot legally use the product is a blocker with a date, and I would prioritize it by the revenue and risk attached. A preference, such as faster performance, is a request to compare on breadth. I would ask each customer for the consequence in their words and verify it: a security review that will fail without admin controls is a blocker; an admin console wish is a preference. Blockers get scheduled by deadline; preferences get ranked with the rest of the roadmap."
- **Q.** How do you protect against concentration and precedent? **A.** "With a rule. Three customers at 12%, 9% and 6% make 27% of revenue, and ranking by their size makes the product theirs. I would ask whether the need is shared by other customers, using a quick survey of the other 197 and the support data. Work that serves several customers goes up; work for one goes through a paid or configurable route. I would tell each customer the decision and the reason, and I would avoid promising a date before engineering confirms scope, because a promise to one large account becomes a precedent for all."

IQ-C15-05 · Hard · execution · EX · Meta

Prompt. Two teams want the same engineers for the next quarter: the growth team, to run experiments to raise new-user engagement, and the integrity team, to reduce harmful content. How do you decide?

Rubric.

- **Baseline:** "Whichever has the bigger numbers."
- **Strong:** Sees that the goals are not in one unit, defines criteria that apply to both, and proposes an allocation.
- **Standout:** Separates constraints (a minimum level of safety) from tradeable goals, uses guardrails and thresholds, sets a review date, and handles the politics with a decision rule and an escalation path.

Model answer. I know the product only from public information. The first thing I'd notice is that the two goals are not in the same unit: growth is measured in engagement, integrity in the prevalence of harmful content. Trading them with one score would be false precision. So I'd separate the question into what is a constraint and what is a choice.

Constraints first. Some integrity work is not tradeable: legal requirements, commitments to regulators, and a minimum safety standard the company has set. I'd ask the integrity team which of their items are in that category. Those are funded first, as constraints, and the growth team cannot trade against them.

What remains is a choice. For the rest, I'd define criteria that apply to both: contribution to the company's goals for the period, size of the expected effect with a range, confidence, cost of delay (what gets worse if we wait), and risk (including reputational and user harm). Cost of delay is important for integrity: a harm that grows each week is different from an experiment that can wait.

Sizing (invented). Say there are 6 engineers. Growth proposes experiments with an expected +0.8% in new-user engagement, a 40% chance each of a positive result. Integrity proposes work to reduce harmful content views from 0.12% to 0.08% of views, a one-third reduction, with higher confidence. These cannot be compared in units, but each can be checked against a threshold the company has set (for example, a target prevalence).

Allocation. Not all-or-nothing. A sensible split: fund integrity to the level that meets the constraint and the target, say 3 engineers, and give 3 to growth, with a guardrail on the growth work (no experiment ships that raises harm metrics beyond a set margin). If the integrity target is not met at 3, move more. Review at the midpoint of the quarter with data.

Decision rule and escalation. If the two leads cannot agree, decide by a rule agreed in advance: constraints first, then expected contribution to company goals weighted by cost of delay. If they still disagree, escalate to the shared leader with a one-page tradeoff table, not a debate.

Why this is not a compromise by default. A 50-50 split has no logic; the split follows from the constraint and the target. What would change my mind: new evidence that the harm is growing fast, which would move engineers to integrity, or a growth opportunity that expires.

Company note. Meta is the setting only: `Inferred` from candidate reports that prioritization prompts at Meta can set growth and integrity goals against each other [`unverified: candidate-reported`]. No Meta data or internal practice is claimed.

Follow-ups.

- **Q.** What is a constraint and what is a choice here? **A.** "Some integrity work is not tradeable: legal obligations, a harm that is severe and rising, and a minimum level of safety agreed with leadership. That sets a floor of engineers who go to the integrity team first. Above the floor, the allocation is a choice, and growth and integrity compete on judgment, not on one score. I would define the floor in advance, in writing, in terms of prevalence thresholds, not engineer counts, and let growth have the rest. Treating integrity as a constraint avoids the false precision of trading a harm reduction against an engagement number."

- **Q.** How do you handle the politics and the review date? **A.** "With a decision rule and an escalation path. The rule: integrity gets the engineers needed to keep prevalence below the agreed threshold; growth gets the remainder, with a stated expected return. Review the split at mid-quarter against the prevalence trend and the growth results, and change it if either moves. If the two leaders disagree, escalate to the person who owns both goals with a one-page note that states the options and the cost of each. A visible rule and date reduce the sense that one team won."

IQ-C15-06 · Hard · execution · EX

Prompt. Option A is a sure, modest win. Option B is a big win that might not work. You have capacity for one. How do you choose?

Rubric.

- **Baseline:** "B has the bigger upside, so B," or "A is safe, so A."
- **Strong:** Compares expected values per unit of effort and considers risk appetite.
- **Standout:** Computes expected value net of cost, shows how a cheap test changes the comparison, treats the probabilities as assumptions, and sequences the options.

Model answer. I'd compare expected value net of cost, then see whether a test changes the picture. Suppose (invented) A is a \$2,000,000 annual benefit with a 60% chance of working, costing 4 person-months at \$25,000 each (\$100,000). B is a \$5,000,000 benefit with a 20% chance, costing 10 person-months (\$250,000).

- A: $0.6 \times \$2,000,000 = \$1,200,000$, less \$100,000, is \$1,100,000.
- B: $0.2 \times \$5,000,000 = \$1,000,000$, less \$250,000, is \$750,000.

On expected value, A is ahead even though B's prize is bigger. Per person-month, A returns about \$300,000 and B about \$100,000.

Does a test change B? The 20% chance is an assumption. Suppose a concierge or prototype test costs 1 person-month (\$25,000) and reliably tells us whether B is viable. If we build B only when the test says yes (20% of the time), the expected value is $0.2 \times \$5,000,000$ less $0.2 \times \$250,000$ for the build, less \$25,000 for the test: $\$1,000,000 - \$50,000 - \$25,000 = \$925,000$. The test improves B from \$750,000 to \$925,000 by avoiding the 80% of builds that would fail. B-with-test is still behind A's \$1,100,000.

Decision. Do A first. Run the cheap test of B in parallel if there is any spare capacity, because the test is worth more than it costs. The choice depends on the probabilities, so I'd state them as assumptions and invite challenge: for B to beat A, its chance of working would need to be above about 27% ($\$1,100,000 + \$250,000 = \$1,350,000$, divided by \$5,000,000 is 27%, ignoring the test).

Beyond the numbers. Risk appetite matters. If the company needs a big bet (a strategic gap, a competitor), B's upside may matter more than expected value, and a portfolio view (several A's and a B) is better than choosing one. And the learning from testing B has option value. What would change my mind: evidence that A's 60% is overstated, or that B's market window is closing.

Follow-ups.

- **Q.** How does a cheap test change the comparison? **A.** "It lets you buy information before paying the full cost. Option B has net expected value of $0.2 \times \$5,000,000 - \$250,000 = \$750,000$, against \$1,100,000 for A. Suppose one person-month (\$25,000) of testing would show whether B works. If B works (20% of the time), you spend the full \$250,000 and gain \$5,000,000; if not, you stop. The expected value of test-then-build is $0.2 \times (\$5,000,000 - \$250,000) - \$25,000 = \$925,000$, much better than building B blind. It is still below A, but the gap has narrowed, and a cheaper test would close it."
- **Q.** How do you treat the probabilities, and how would you sequence the options? **A.** "As assumptions I would challenge: where did 60% and 20% come from? If they come from past launches of similar size, they are evidence; if they come from the team's optimism, I would discount them. I would run the sensitivity: at what probability does B beat A? Here B needs about 26% (so that $0.26 \times 5,000,000 - 250,000 = \$1,050,000$). Sequence: build A now, use a small slice of capacity to test B's riskiest assumption, and decide on B next quarter with evidence."

IQ-C15-07 · Hard · execution · EX

Prompt. A senior executive wants you to build their idea next. The data suggests another item is better. How do you handle it?

Rubric.

- **Baseline:** "Do what the executive says," or "refuse and cite the data."
- **Strong:** Treats it as a decision with evidence, brings options and a recommendation, and uses a respectful script.
- **Standout:** Seeks the need behind the idea, separates what the data shows from what it cannot, proposes a cheap test, offers a way to include the idea, and keeps the relationship and the decision rights clear.

Model answer. I'd begin by assuming the executive knows something I don't: a customer conversation, a strategic bet, a competitor's move. So the first step is to ask about the need behind the idea, not to defend my list. "Help me understand what you're seeing that makes this the priority." Often the idea is a solution to a problem that can be solved in more than one way.

Show the tradeoff, not just my view. I'd bring the options with their costs. Say the executive's idea is X and my data points to Y. A one-page table: expected impact on the agreed goal with a range, effort, confidence, and what each displaces. "If we build X, we defer Y, which we estimate at \$A a year; X's value depends on assumption Z, which we have not tested."

Be honest about the data's limits. Data on past behavior does not capture a new strategic bet. If the executive's case is strategic, my numbers on near-term impact are not the whole argument. I'd say what the data does and does not show.

Offer routes that respect the executive. (1) A cheap test of X's riskiest assumption in two weeks, with success and kill criteria agreed beforehand; (2) a smaller version of X that fits alongside Y; (3) a sequence, Y now and X next quarter with a date. These let the executive see that their idea was taken seriously.

Use the script (FW-C15-07). "I see the need: [need]. To build X now we'd delay Y, which means [consequence]. What I can do is [test or smaller version]. If the test shows [result], I'd move X up." And keep the decision rights clear: if they decide to proceed after seeing the tradeoff, I'd do it well and record the assumption so we learn from the outcome, without saying "I told you so."

What I'd avoid. Hiding the tradeoff to avoid conflict, quietly deprioritizing, or turning it into a contest of status. What would change my mind: the executive revealing a fact (a commitment, a customer deadline) that changes the value of X.

Follow-ups.

- **Q.** How do you show the tradeoff and not just your view? **A.** "With a shared one-page comparison: the executive's idea and my top item, each with the goal it serves, reach, impact estimate, confidence, cost and what we would learn. I would ask them to check the inputs and tell me where I am wrong. Often the difference is in a single assumption, and naming it makes the conversation about evidence. I would present it as the decision they own, with my recommendation, and avoid framing it as the data against their idea."
- **Q.** What cheap test or compromise would you propose? **A.** "A time-boxed test of the executive's idea, such as a prototype or a manual version for a small group, with a threshold agreed in advance, while my top item proceeds. If the test passes, the idea moves up with evidence; if not, the executive sees it. Another option is a smaller version of the idea that fits beside the main item. I would keep the relationship by taking their need seriously and by being clear who decides: if they decide to proceed regardless, I would commit, record the risk and the signal that would reopen it."

IQ-C15-08 · Intermediate · execution · EX

Prompt. A feature used by 4% of weekly users takes about one and a half engineers to maintain. Should you retire it?

Rubric.

- **Baseline:** "Yes, it's a small share," or "no, someone uses it."
- **Strong:** Looks at who uses it, the cost, and the alternatives.
- **Standout:** Quantifies the cost against the revenue at risk, finds the break-even churn, considers lighter options (maintenance mode, simplification, migration), plans communication and a reversible rollout, and sets a decision rule.

Model answer. *Who uses it, and why.* 4% of 2 million weekly users is 80,000 people. Share of users says little by itself: what matters is who they are. Are they heavy, loyal or high-value users (suppose their retention is 92% against 70% overall)? Do they use it for a job nothing else does? Are they concentrated in a segment we care about?

Cost against what is at risk (invented figures). At about \$15,000 a month per engineer, one and a half engineers cost \$22,500 a month, plus the opportunity cost of what they would otherwise build. If these users pay about \$4 a month and some leave when the feature goes, the revenue at risk is the number who churn times \$4. The break-even is $22,500 \div (80,000 \times 4) =$ about 7.0% of those users leaving. If 5% leave, retiring saves \$22,500 and loses \$16,000, a net gain of \$6,500 a month; if 10% leave, it loses \$32,000, a net loss of \$9,500; at 20%, a net loss of \$41,500. So the decision turns on the churn we would cause, which I can estimate rather than guess.

Options between keep and kill. (A) Keep as is. (B) Put it in maintenance mode: freeze features, fix only critical bugs, and cut the cost to perhaps half an engineer. (C) Simplify or fold its core into another feature. (D) Retire with a migration path, a long notice period and an export. (E) Retire outright.

How I'd estimate churn. Look at what these users do instead when the feature is unavailable (a holdout or a past outage), survey the heaviest users, and test a deprecation notice with a small group to see who leaves. Also check dependencies: other features, integrations and support load.

Recommendation. Start with B and measure; if the estimated churn from retirement is under about 5% and the freed capacity has a clear, better use, move to D with notice and migration. If the users are high-value and the churn estimate is above the break-even, keep it in maintenance mode and revisit when the cost or use changes. I'd decide with a rule agreed in advance and tell users plainly what is happening and why.

Signals. Who uses it; cost against revenue at risk; a computed break-even; options between keep and kill; communication and reversibility.

Follow-ups.

- **Q.** What if the 4% are your most loyal, highest-retention users? **A.** "Then the revenue at risk is the loyalty of the group that sustains the product, not just their monthly payment, so I'd weight their retention and referrals. I'd lean toward keeping the feature in maintenance mode, learn what job it does for them and whether another feature could serve it, and only retire it with a replacement or a very clear migration. A small share of users can be a large share of the product's value."
- **Q.** How would you communicate the retirement? **A.** "Early, plainly and with a path: say what is going away, when, why, and what to do instead, with an export of their data and a migration guide. I'd give a long notice period (months, not weeks for an important feature), reach users in product and by email, offer support for the heaviest users, and keep a feedback channel open. I'd track complaints and churn after the notice as the guardrail, and I'd be ready to extend the date if the migration path is not working."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C15-01 · Compute a RICE ranking

Problem. Score and rank: W (reach 50,000, impact 1, confidence 80%, effort 2); X (20,000, 3, 60%, 3); Y (80,000, 0.5, 90%, 2); Z (10,000, 2, 100%, 1). Then say what would change the order.

Context. A team is choosing two features.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C15-03 and the sensitivity check.

Solution. $W = 50,000 \times 1 \times 0.8 \div 2 = 20,000$. $X = 20,000 \times 3 \times 0.6 \div 3 = 12,000$. $Y = 80,000 \times 0.5 \times 0.9 \div 2 = 18,000$. $Z = 10,000 \times 2 \times 1 \div 1 = 20,000$. Ranking: W and Z tie at 20,000, then Y at 18,000, then X at 12,000. Z's 100% confidence is suspicious; at 70% its score is 14,000 and it drops below Y. X's confidence at 100% would raise it to 20,000, a tie for first. The order of the top four depends on confidence estimates, so I'd find evidence for those before deciding.

Alternative solution. Report the ranking as bands: top tier {W, Z, Y}, then X, and use the goal to break the tie.

Tradeoffs. Scores to the unit imply a precision that the inputs do not have.

EX-C15-02 · Same list, two goals

Problem. The options are: a referral program, a faster checkout, an improved search and a loyalty program. Say how the ranking would differ under two goals: (a) grow new users; (b) increase repeat purchases by existing customers.

Context. The company has to choose its goal for the quarter.

Data. None; reason qualitatively and name the reach and impact that each goal implies.

Constraints. Six minutes.

Expected approach. Derive criteria from each goal (FW-C15-04).

Solution. Under (a), reach is people not yet users: the referral program ranks high (its mechanism is reaching new people); search and loyalty barely matter; checkout matters to new users only at their first purchase. Under (b), reach is existing customers: the loyalty program and search improvements rise (they bring people back and make repeat shopping easier), checkout rises (friction on repeat purchase), and the referral program drops. The same four options produce nearly opposite orders, which is why the goal comes before the scoring.

Alternative solution. Choose the option that is second under both goals if the company must serve both, and say what it gives up.

Tradeoffs. A compromise pick can serve neither goal well.

EX-C15-03 · A weighted scorecard and flip test

Problem. Options A (impact 5, reach 3, confidence 4, ease 2), B (3, 5, 3, 4), C (4, 4, 2, 3), each scored 1 to 5. Weights 1: impact 40%, reach 30%, confidence 20%, ease 10%. Weights 2: 20%, 20%, 20%, 40%. Compute both rankings and state what is robust.

Context. A flip test.

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C15-04.

Solution. Weights 1: $A = 0.4 \times 5 + 0.3 \times 3 + 0.2 \times 4 + 0.1 \times 2 = 2.0 + 0.9 + 0.8 + 0.2 = 3.9$. $B = 1.2 + 1.5 + 0.6 + 0.4 = 3.7$. $C = 1.6 + 1.2 + 0.4 + 0.3 = 3.5$. Order A, B, C. Weights 2: $A = 1.0 + 0.6 + 0.8 + 0.8 = 3.2$. $B = 0.6 + 1.0 + 0.6 + 1.6 = 3.8$.

$C = 0.8 + 0.8 + 0.4 + 1.2 = 3.2$. Order B first, A and C tied. So the top choice flips between A and B when ease matters more: the decision depends on how much the team values ease against impact. That is the tradeoff to discuss. B is the more robust second-or-first choice, since it is first or second in both.

Alternative solution. Choose B if the team is unsure of its weights, as the choice that is never worse than second.

Tradeoffs. A robust option is rarely the best under any one weighting.

EX-C15-04 · Expected value per person-month

Problem. P: \$3,000,000 at 50%, 5 person-months. Q: \$10,000,000 at 15%, 12 person-months. R: \$1,000,000 at 90%, 1.5 person-months. Compute expected value and value per person-month, and rank.

Context. Capacity of 6 person-months.

Data. As above.

Constraints. Five minutes.

Expected approach. C15.5.

Solution. P: EV \$1,500,000, \$300,000 per person-month. Q: EV \$1,500,000, \$125,000 per person-month. R: EV \$900,000, \$600,000 per person-month. By value per person-month: R, P, Q. With 6 person-months: R (1.5) and P (5) total 6.5, slightly over; R plus part of P does not work. So fund P (5) and have 1 left, or R (1.5) and 4.5 for something else. A sensible plan: R first (cheap, high return), then P if capacity can be stretched or the date moved. Q has the highest prize (\$10,000,000) and the lowest return per unit; test its 15% before committing.

Alternative solution. If the company needs a large bet, include Q with a test first.

Tradeoffs. The probabilities are assumptions; the ranking changes if Q's chance is 30%.

EX-C15-05 · A shorter deadline

Problem. Three engineers have 6 weeks for 9 stories (about 2 person-weeks each). The date moves to 3 weeks. Compute the capacity and the number of stories; then compute the effect of adding one engineer (25% productive in week 1, 50% in week 2, full in week 3; the three existing lose 20% of two weeks to onboarding).

Context. The sponsor suggests "just add someone."

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C15-05.

Solution. At 3 weeks: $3 \times 3 = 9$ person-weeks, 4.5 stories. The new engineer delivers $0.25 + 0.5 + 1 = 1.75$ person-weeks; onboarding costs $3 \times 2 \times 0.2 = 1.2$; net gain 0.55, for 9.55 person-weeks, about 4.8 stories. Adding a person adds roughly a third of a story in this window, while communication paths rise from 3 to 6. So the lever is scope: choose the 4 or 5 stories that deliver the core, defer the rest to a follow-up, or move the date.

Alternative solution. Assign the new engineer to a separable task that does not need onboarding in the core code, for example test tooling, and say it will help later, not now.

Tradeoffs. Adding people early in a long project can pay; late in a short one it rarely does.

EX-C15-06 · Design an MVP test

Problem. An app wants to add paid expert coaching. The riskiest assumption is that users will pay for it. Design an MVP test with success and kill criteria, and compute the confidence intervals for three outcomes with 100 offers: 30, 22 and 15 purchases.

Context. Engineering says it would take five months to build.

Data. As above.

Constraints. Seven minutes.

Expected approach. FW-C15-06 and C14.5.

Solution. A fake-door test: show a "Book a coach, \$30" button to 100 eligible users; when they click, ask them to pay and book a time, and fulfill manually with a coach. Success: at least 30 of 100 buy (30%). Kill: 15 or fewer (15%). Intervals (95%): 30 of 100 is about 22% to 40%; 22 of 100 about 15% to 31%; 15 of 100 about 9% to 23%. So 30 clears the bar with an interval whose low end is above the kill bar; 22 straddles and calls for more data or a redesign of the offer; 15 is at the kill bar, with an upper end of 23%. Guardrails: refund requests and the coaches' time. Cost: two weeks and a few coaching sessions, against five months of build.

Alternative solution. Test willingness to pay with a pre-order instead of a purchase, which is cheaper but weaker evidence.

Tradeoffs. A manual service does not test whether it can scale, so scalability is the next assumption to test if this passes.

EX-C15-07 · Fix a MoSCoW list

Problem. A release list has 14 items, all marked "must." Capacity is 60% of the effort. Describe how to fix it.

Context. A planning meeting.

Data. None.

Constraints. Five minutes.

Expected approach. IQ-C15-02.

Solution. Redefine "must" as an item without which the release fails its goal or cannot ship by the date (legal, core flow). Apply the test to each item. Then compare the remaining total with capacity (60%) and ask what leaves. Move items to "should," "could" and "won't this time," each with a reason, and date the "won't" items for review. Show options with costs where stakeholders disagree. The outcome should be a list whose "musts" fit capacity, with a buffer, and a written reason for each deferral.

Alternative solution. Ask each stakeholder to rank their own top three, then compare across stakeholders, since people will give up their fourth item more readily than their first.

Tradeoffs. A strict "must" reduces conflict at the meeting and requires follow-up communication.

EX-C15-08 · Write the no

Problem. Write a four-part reply (FW-C15-07) to each: (a) a salesperson wants a custom report for a prospect before the quarter ends; (b) a designer wants three more weeks to polish a feature; (c) an executive wants a feature added to this release.

Context. Each request is reasonable and does not fit the plan.

Data. None.

Constraints. Six minutes.

Expected approach. The script: need, tradeoff, alternative, trigger.

Solution. (a) "I see the deal depends on seeing the data. A custom build would delay the reporting upgrade other customers wait for. What I can do is a manual export for this prospect this week, and a general version next quarter. If more than two prospects ask, I'll move it up." (b) "I see the polish matters for the first impression. Three more weeks would push the launch past the event."

I'd propose polishing the first-run screens and the payment path now and scheduling the rest for the next release with a date. If testing shows the rough areas hurt completion, we'll move them earlier." (c) "I see why it matters. Adding it means dropping X or moving the date by two weeks. Which do you prefer? A smaller version could ship this release, and the full version in the next."

Alternative solution. Attach the cost of each option to the reply so the person can choose.

Tradeoffs. Offering alternatives takes more effort than refusing, and it preserves relationships.

EX-C15-09 · Break-even effort

Problem. Feature F has reach 40,000, impact 1, confidence 80%, effort 2 person-months. A competing feature scores 12,000. At what effort does F tie, and what do you conclude?

Context. A sensitivity check on effort.

Data. As above.

Constraints. Four minutes.

Expected approach. FW-C15-08.

Solution. F's score is $40,000 \times 1 \times 0.8 \div 2 = 16,000$. It ties 12,000 when effort E satisfies $40,000 \times 0.8 \div E = 12,000$, so $E = 32,000 \div 12,000 \approx 2.67$ person-months. F stays ahead unless effort is more than about a third higher than estimated. Since engineering estimates are often low, that margin is thin: I'd ask engineering for a range, check the riskiest part of the estimate, and treat the ranking as fragile.

Alternative solution. Find the break-even for confidence: F ties at confidence 60% ($40,000 \times c \div 2 = 12,000$ gives $c = 0.6$).

Tradeoffs. Knowing the break-even tells you which number to spend time on.

EX-C15-10 · A timed full answer

Problem. Give a 20 to 25 minute answer to: "You are the PM of a note-taking app. You can build one of three things this quarter: collaboration (share and edit with others), a web clipper, or offline mode. Which first?"

Context. Self-scored; record yourself. State assumptions aloud.

Data. None. Invent only what you label.

Constraints. Use the 25-minute budget in section 4, compressed.

Expected approach. The full spine (FW-C15-01), scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Ask for the goal; assume growth in weekly active users among students. Criteria: reach among the target users, impact on weekly activity, confidence, effort, strategic fit (collaboration creates a network effect). Size each with a chain and a range: offline reaches users with poor connectivity, which matters if the target includes them (C09); the clipper reaches users who gather research; collaboration reaches pairs and groups and spreads the product to non-users. Score, then change the least certain number (collaboration's effort is the largest and the most uncertain). Name the riskiest assumption (students will invite classmates) and a cheap test (a share-a-note link that does not need full collaboration). Decide, say what you will not do and what would change the order (a different goal, such as retention among existing users).

Alternative solution. Pick the web clipper as a quick win and use the quarter to test collaboration demand, if the team is small.

Tradeoffs. The biggest strategic bet carries the biggest uncertainty; a cheap test lets you learn before committing.

13. Practice protocol

Timed drills (three a week for four weeks).

- *Week 1: goal and criteria.* Use IQ-C15-01, IQ-C15-02, EX-C15-02 and EX-C15-07. State the goal before any ranking, and derive criteria aloud.
- *Week 2: sizing and sensitivity.* Use EX-C15-01, EX-C15-03, EX-C15-04 and EX-C15-09. Do the arithmetic by hand, find the break-even, and say whether the ranking is robust.
- *Week 3: tradeoffs and tests.* Use IQ-C15-03, IQ-C15-06, IQ-C15-08, EX-C15-05 and EX-C15-06. Name the fixed point and the lever, compute expected value, and design an MVP with criteria set first.
- *Week 4: conflict and full rounds.* Use IQ-C15-04, IQ-C15-05, IQ-C15-07, EX-C15-08 and EX-C15-10. Practice the four-part no, a resource conflict, and two full 25-minute answers with a partner who changes the goal midway.

With a partner. Ask your partner to say "Why not do both?", "Where did that number come from?" and "The VP wants the other one," at random. Practice returning to the goal in one sentence.

Keep a decisions log. For each drill, write the goal, the top choice, what flips it and what you deferred. After eight drills, check whether you always named the opportunity cost.

Feedback checklist (yes, partly, no).

1. Did I state the goal and constraints before ranking?
2. Did I derive criteria from the goal and say how I weighted them?
3. Did I size options with a chain and a range?
4. Did I run a sensitivity check and name what flips the ranking?
5. Did I name the opportunity cost and what we are not doing?
6. Did I identify the riskiest assumption and a cheap test with criteria set first?
7. Did I handle the tradeoff among scope, time, quality and resources?
8. Did I say no with a reason, an alternative and a trigger?
9. Did I decide, and say what would change my mind?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the prompt, the goal, the criteria, the decision, the sensitivity result, your score out of ten and the step you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Take your team's current backlog or roadmap. Write the goal it serves in measurable terms and the constraints. For the top five items, write the criteria they should meet for that goal, and mark which items would not survive.

Week 1. Rescore the top five with a simple scorecard derived from the goal (FW-C15-04), with ranges. Run the flip test (change the weights) and note where the order changes. Share the result with your manager and the lead engineer, and ask what is missing.

Week 1. Find one item whose value depends on an untested assumption. Write the assumption map (FW-C15-06) and the cheapest test with success and kill criteria, before anyone builds.

Week 2. Pick one request you plan to decline or defer, and write the four-part reply (FW-C15-07). Deliver it in person, and note the reaction and what you learned about the need.

Week 2. Compute the opportunity cost of one planned item: what the same capacity could build instead and its estimated value.

Week 3. Write a one-page "not now" list for the quarter, with a reason and a revisit date for each item. Share it with stakeholders before the quarter starts.

Deliverables that become stories: the goal-derived scorecard with a flip test (a time you made a priority decision defensible), the assumption map and test (a time you avoided building the wrong thing), and the not-now list with reasons (a time you said no well). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. Prioritization is a decision about a goal under limited capacity: state the goal and constraints, derive criteria from it, size the options with ranges, and name what you give up (the opportunity cost).
2. Use frameworks such as RICE to organize the numbers, not to make the decision; define reach and impact against the goal, and always run a sensitivity check to see what flips the ranking.
3. Tradeoffs are among scope, time, quality and resources; name what is fixed, pull the lever with the least cost (usually scope), and do not assume people or quality substitute for time.
4. Find the riskiest assumption and test it cheaply with a minimum viable product, with success and kill criteria written first.

5. Say no with the need, the tradeoff, an alternative and a trigger to revisit, and treat what you stop doing as a decision as important as what you start.

Further reading

- Marty Cagan, *Inspired* (2nd edition, 2017). On choosing and testing what to build, and on the role of discovery.
- Donald G. Reinertsen, *The Principles of Product Development Flow* (2009). Cost of delay and queueing [V: Reinertsen, Celeritas Publishing, 2009 (WorldCat)].
- Eric Ries, *The Lean Startup* (2011). The minimum viable product and validated learning [V: Crown Business, 13 September 2011, ISBN 9780307887894].
- Sean McBride, Intercom's description of the RICE scoring model (blog; dated 5 January 2018 on Intercom's site; accounts of when RICE was first used at Intercom range from 2015 to 2017) [V: Intercom blog, "RICE: Simple prioritization for product managers" by Sean McBride; dated 5 January 2018 per the site and a content aggregator; the year RICE was first used is not established].
- Frederick P. Brooks, Jr., *The Mythical Man-Month* (anniversary edition, 1995). On adding people to late projects [V: Brooks, *The Mythical Man-Month*, 1975 with later editions 1982 and 1995].
- Daniel Kahneman, *Thinking, Fast and Slow* (2011). The planning fallacy and the outside view.
- Annie Duke, *Thinking in Bets* (2018). Decisions under uncertainty [V: Portfolio, 6 February 2018, ISBN 9780735216358].
- Bill Coughran, "More wood behind fewer arrows," Official Google Blog (20 July 2011; googleblog.blogspot.com/2011/07/more-wood-behind-fewer-arrows.html), and Verne Kopytoff, "Is this what a 'focused' Google looks like?" *Fortune* (3 July 2012). The sources for CS-C15-02; Larry Page's earnings-call remarks of 14 July 2011 are known through press reports.